

# Operating Systems: Process Management & Scheduling

## 1.1 Process Lifecycle and Context Switching

A process is a program in execution. The operating system manages processes through a Process Control Block (PCB) containing process state, program counter, CPU registers, and scheduling information.

When switching the CPU to another process, the kernel saves the state of the old process and loads the saved state for the new process; this mechanism is known as a Context Switch. Context switches represent pure overhead because the system does no useful computational work while switching.

## 1.2 CPU Scheduling Algorithms

The CPU scheduler selects from among the processes in ready memory and allocates CPU time to one of them.

- **First-Come, First-Served (FCFS): Non-preemptive scheduling. Simplest to implement, but suffers from the** Convoy Effect where short processes wait behind one long CPU-bound process, resulting in poor average wait time.
- **Shortest Job First (SJF): Selects the process with the shortest next CPU burst. Provably optimal in minimizing** average waiting time. However, predicting the length of the next CPU burst accurately in advance is impossible.
- **Round Robin (RR): Preemptive algorithm designed for time-sharing systems. Each process is allocated a fixed time** quantum (time slice), typically 10 to 100 milliseconds. If the quantum is too large, RR degenerates to FCFS. If the quantum is too small, context switch overhead severely degrades overall CPU efficiency.
- **Priority Scheduling: CPU is allocated to the process with highest priority. A critical problem is Starvation** (indefinite blocking), where low-priority processes never execute. The standard solution is Aging, gradually increasing the priority of processes that wait in the ready queue for prolonged periods.

# Operating Systems: Memory Management & Virtual Memory

## 2.1 Virtual Memory and Address Translation

Virtual memory separates user logical memory from physical memory. This allows an execution of processes that are not completely loaded in physical RAM, providing the illusion of very large address spaces.

The Memory Management Unit (MMU) translates logical addresses generated by the CPU into physical addresses.

Logical memory is divided into fixed-size blocks called pages, while physical memory is divided into fixed-size blocks of the identical size called frames (commonly 4KB in modern architectures).

## 2.2 Page Tables and Translation Lookaside Buffer (TLB)

Every process possesses a Page Table to map page numbers to frame numbers. Accessing memory through a page table in RAM requires two memory accesses per data lookup. To accelerate translation, modern CPUs utilize a hardware cache called the Translation Lookaside Buffer (TLB). A TLB hit retrieves the frame number in a single clock cycle, while a TLB miss forces a page table lookup in main memory.

## 2.3 Page Faults and Page Replacement Algorithms

When a process accesses a page not currently mapped in physical RAM, the hardware raises a Page Fault trap to the OS. The OS retrieves the desired page from secondary storage (swap space) and loads it into a free frame.

- **First-In-First-Out (FIFO):** Replaces the oldest loaded page. May exhibit Belady's Anomaly, where increasing the number of physical page frames paradoxically results in an increased number of page faults.
- **Least Recently Used (LRU):** Replaces the page that has not been used for the longest period of time. Approximates optimal replacement and never suffers from Belady's Anomaly.
- **Thrashing:** A condition where a system spends more time servicing page faults and paging than executing actual instructions. Occurs when the sum of process working sets exceeds total physical memory capacity.

# Operating Systems: Deadlocks & Process Synchronization

## 3.1 The Four Coffman Conditions for Deadlock

A deadlock is a situation wherein every process in a set is waiting for an event that only another process in the same set can cause. A deadlock can arise if and only if all four Coffman conditions hold simultaneously:

- 1. Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
- 2. Hold and Wait:** A process must be currently holding at least one resource and requesting additional resources that are being held by other operating processes.
- 3. No Preemption:** Resources cannot be forcibly taken; they can only be released voluntarily by the process.
- 4. Circular Wait:** A closed chain of processes exists such that each process holds at least one resource needed by the next process in the chain.

## 3.2 Deadlock Handling Strategies

- **Deadlock Prevention:** Eliminates at least one of the four Coffman conditions (e.g., imposing total resource ordering).
- **Deadlock Avoidance:** Requires the system to possess a priori information regarding resource requests.  
The Banker's Algorithm evaluates each allocation to ensure the system remains in a Safe State.  
A state is safe if there exists a safe sequence of processes that can all finish without deadlock.
- **Deadlock Detection and Recovery:** Allows deadlocks to occur, periodically invokes a cycle-detection algorithm on the Resource Allocation Graph (RAG), and recovers via process termination or resource preemption.

## 3.3 Synchronization Primitives: Mutex vs Semaphore

A Mutex is a locking mechanism used to synchronize access to a resource; it possesses ownership (only the thread that locked it can unlock it). A Counting Semaphore is a signaling mechanism with an integer counter (`wait() / P()` and `signal() / V()`); any thread can signal a semaphore to unblock waiting threads.

# Operating Systems: File Systems & Storage Management

## 4.1 File Allocation Methods

The OS manages how storage blocks are allocated to files on secondary storage:

- **Contiguous Allocation:** Each file occupies a set of contiguous blocks on disk. Provides stellar sequential and direct access performance, but suffers severely from External Fragmentation and file size inflexibility.
- **Linked Allocation:** Each file is a linked list of disk blocks. Eliminates external fragmentation, but does not support efficient direct random access (finding block N requires traversing N-1 pointers).
- **Indexed Allocation:** Brings all pointers together into an Index Block (e.g., Inodes in UNIX systems).  
UNIX Inodes contain direct block pointers, single indirect pointers, double indirect pointers, and triple indirect pointers to support enormous file sizes while preserving fast access for small files.

## 4.2 Disk Scheduling Algorithms

The operating system schedules I/O disk requests to minimize head seek time (the dominant delay in magnetic storage):

- **FCFS (First-Come, First-Served):** Service requests in incoming order. Fair, but incurs excessive arm movement.
- **SSTF (Shortest Seek Time First):** Selects the request with minimum seek time from the current head position.  
Substantially reduces seek time compared to FCFS, but causes starvation for distant cylinder requests.
- **SCAN (Elevator Algorithm):** Disk arm starts at one end and moves toward the other end, servicing requests along the way until it reaches the opposite edge, where it reverses direction.
- **C-SCAN (Circular SCAN):** Moves head in only one direction servicing requests; once it reaches the end, it immediately returns to the beginning without servicing any requests on the return trip, providing uniform wait times.